

Max Heap vs Linear Search – Student Scores

Implementation, execution, trace tables, complexity analysis, comparison and conclusion

1. Given Data

Student scores: **78, 92, 65, 88, 95, 72, 84, 90**.

The objective is to build a Max Heap, record the heap after each insertion, find the highest score using Max Heap and Linear Search, count operations/comparisons, and compare their performance.

2. Max Heap Insertion Trace

Step	Inserted	Heap after insertion	Comparisons / swaps
1	78	78	0 / 0
2	92	92, 78	1 / 1
3	65	92, 78, 65	1 / 0
4	88	92, 88, 65, 78	2 / 1
5	95	95, 92, 65, 78, 88	2 / 2
6	72	95, 92, 72, 78, 88, 65	2 / 1
7	84	95, 92, 84, 78, 88, 65, 72	2 / 1
8	90	95, 92, 84, 90, 88, 65, 72, 78	3 / 2

Final Max Heap: **[95, 92, 84, 90, 88, 65, 72, 78]**. The root contains the largest value.

3. Finding the Maximum Using Max Heap

The maximum is stored at the root. Therefore, the highest score is **95**. It can be obtained directly without scanning the other elements.

4. Finding the Maximum Using Linear Search

Step	Score	Current maximum	Comparison
1	78	78	Initial
2	92	92	$92 > 78$
3	65	92	$65 < 92$
4	88	92	$88 < 92$
5	95	95	$95 > 92$
6	72	95	$72 < 95$
7	84	95	$84 < 95$
8	90	95	$90 < 95$

Linear search makes **$7 = n - 1$** comparisons for $n = 8$ scores.

5. C Program

```
#include <stdio.h>
#define MAX 100

int heap[MAX], size = 0;

void insert(int value)
{
    int i, parent, temp;
    size++;
    i = size - 1;
    heap[i] = value;

    while (i > 0)
    {
        parent = (i - 1) / 2;
        if (heap[parent] >= heap[i])
            break;

        temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;
        i = parent;
    }
}

void displayHeap()
{
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", heap[i]);
    printf("\n");
}

int linearMax(int a[], int n, int *comparisons)
{
    int i, max = a[0];
    *comparisons = 0;

    for (i = 1; i < n; i++)
    {
        (*comparisons)++;
        if (a[i] > max)
            max = a[i];
    }
    return max;
}

int main()
{
    int scores[] = {78, 92, 65, 88, 95, 72, 84, 90};
    int n = 8, i, max, comparisons;

    printf("MAX HEAP INSERTION\n");
    for (i = 0; i < n; i++)
    {
        insert(scores[i]);
        printf("After inserting %d: ", scores[i]);
        displayHeap();
    }

    printf("\nMAXIMUM USING MAX HEAP\n");
    max = heap[0];
    printf("Highest score = %d\n", max);
    printf("Comparisons = 0\n");

    printf("\nMAXIMUM USING LINEAR SEARCH\n");
    max = linearMax(scores, n, &comparisons);
    printf("Highest score = %d\n", max);
    printf("Comparisons = %d\n", comparisons);

    return 0;
}
```

6. Sample Execution Output

```
MAX HEAP INSERTION
After inserting 78: 78
After inserting 92: 92 78
After inserting 65: 92 78 65
```

After inserting 88: 92 88 65 78
 After inserting 95: 95 92 65 78 88
 After inserting 72: 95 92 72 78 88 65
 After inserting 84: 95 92 84 78 88 65 72
 After inserting 90: 95 92 84 90 88 65 72 78

MAXIMUM USING MAX HEAP
 Highest score = 95
 Comparisons = 0

MAXIMUM USING LINEAR SEARCH
 Highest score = 95
 Comparisons = 7

7. Performance Comparison

Operation	Max Heap	Linear Search
Find maximum	$O(1)$, root access	$O(n)$
Insert new score	$O(\log n)$	$O(1)$ if simply appended
Repeatedly maintain maximum	$O(\log n)$ insertion + $O(1)$ query	$O(n)$ per independent maximum search
Space	$O(n)$ heap storage	$O(1)$ extra space
Given data	95; root access	95; 7 comparisons

8. Effect of Increasing Number of Students

As the number of students increases, linear search needs to inspect more scores, so finding the maximum grows as $O(n)$. In a Max Heap, the maximum remains at the root, so finding it remains $O(1)$. Each new score can be inserted in $O(\log n)$.

9. Time and Space Complexity

Operation	Time	Space
Max Heap insertion	$O(\log n)$	$O(n)$ heap storage
Max Heap find maximum	$O(1)$	$O(n)$ heap storage
Linear search maximum	$O(n)$	$O(1)$ extra
Repeated insertion build	$O(n \log n)$	$O(n)$

10. Final Conclusion

The highest score is **95**. For the given eight scores, Max Heap finds the maximum directly from the root, while linear search needs 7 comparisons. For an application that continuously adds student scores and repeatedly needs the current highest score, a Max Heap provides $O(\log n)$ insertion and $O(1)$ maximum access. If the list is fixed and the maximum is needed only once, linear search is simpler. The appropriate choice therefore depends on the update and query pattern; for continuous highest-score maintenance, the Max Heap structure matches the required operations well.

11. GitHub Repository Structure

File / Folder	Content
src/max_heap.c	C implementation of Max Heap insertion and maximum
src/linear_search.c	C implementation of linear-search maximum
input/input.txt	78 92 65 88 95 72 84 90
output/output.txt	Execution output
trace_table/trace_table.xlsx	Heap insertion and search trace tables
analysis/complexity_analysis.txt	Time and space complexity

analysis/comparison_table.txt	Max Heap vs Linear Search
conclusion/final_conclusion.txt	Final conclusion
README.md	Assignment overview

Replace the repository placeholder with your actual GitHub repository URL after uploading the files.